Graphic Novel Price Tracker

Joshua Alcaraz, Gray Espinoza, and Jairo Santos

Introduction

We wanted a web app that allowed users to track the current and historic prices of graphic novels against the online retailers: Amazon, Barnes & Noble, CheapGraphicNovels, InStockTrades, and OrganicPricedBooks. It would provide a similar user experience to Deku Deals[1] which tracks the prices of video games across the current generation of consoles and links to retailer listings. The purpose of our web app is the facilitation of bargain shopping of comic book collected editions.

[1] dekudeals.com

Software Overview

We would accomplish our purpose via the implementation of four main components: our retail scrapers implemented with Playwright, a SQLite database, a FastAPI API, and a React-based front-end.

Given an ISBN and, optionally, a title, our scrapers would return a retailer name, listing title, current price, and URL of the listing. This would repeat every four hours for a fixed 421 ISBNs across our five retailers. However, our database only stores the most recent scrape for a given day. The API would then solely expose our database to our front-end where our users could search via a catalog, ISBN, or title with the option of viewing retailer listings.

Testing Plan

Grey: Validate that all scrapers return expected dataclass on pushes.

Jairo: Validate every function in `dbqueries.py`:

- get_prices_for_book(search_term: str), returns the current prices of a graphic novel given an ISBN or title
- best_deals(limit: int), returns the best deal for a graphic novel given an ISBN or title
- price_history(isbn), returns the price history of a graphic novel

Test Cases (Grey)

Function: scrape(isbn: int, title: str)

Functionality Test: either passes scraped data or blank data - PASSED

Boundary Test: passes appropriate dataclass with blank data - PASSED

Error Handling Test: passes appropriate dataclass with blank data - PASSED

Test Cases (Jairo)

Functionality Test:

- get_prices_for_book(), verifies retrieval of graphic novel and its price
- best_deals(), verifies structure of record mapping
- price_history(), verifies that prices are displayed chronologically through tracked dates

Boundary Test:

- get_prices_for_book(), verifies outputs for extreme inputs
- best_deals(), tests the lower bounds of pagination/requests
- price_history(), tests a graphic novel with no history

Error Handling Test:

- get_prices_for_book(), tests against null or incomplete data
- best_deals(), makes sure that the API handles malformed strings smoothly
- price_history(), verifies that the API rejects bad data before it reaches the database

Test Results (Jairo)

Functionality Test:

- get_prices_for_book(), confirmed that ISBN exists and maps title to matching dictionary response payload (200 OK) - PASSED
- best_deals(), high volume records return a list with valid mapped items - PASSED
- price_history(), queried graphic novel with a history length >= 2 should show up on a timeline - PASSED

Boundary Test:

- get_prices_for_book(), querying an invalid 13 digit ISBN structures an empty array (200 OK) instead of crashing - PASSED
- best_deals(), lower bounds of 0 return an empty array without returning an internal error - PASSED
- price_history(), graphic novel with no history returns an empty array layout without crashing timeline graph - PASSED

Error Handling Test:

- get_prices_for_book(), rejects a NULL price value and verifies that the API omits correctly with length 0 to keep front-end layout the same - PASSED
- best_deals(), malformed string (?limit=not-an-integer) is blocked and raises a 422 Unprocessable Entity message - PASSED
- price_history(), invalid string in ISBN search is blocked with 422 Unprocessable Entity message - PASSED

Recommendations

Grey: No recommendations.

Jairo:

- Rather than filtering out NULL/0 values during requests, we can add a NOT NULL constraint to keep it from entering in the first place
- We can speed up the search of title and ISBN using a b-tree index. It will serve as something like a lookup table.

Demo

A demo is available at github.com/CSUF-CPSC362-2026Su/graphic-novel-price-tracker under the web submodule's deployments.

Conclusion and Future Work

Our group was able to successfully implement all our main components and requirements such as the catalog view, search functionality, price history timeline, and individual item listings.

In the future, we plan to polish the data scraping process, scrape more metadata, allow for user submission into our scraping list, add more cover images, speed up load times, and tidy the overall front-end.